

Rapport individuel — Itération 1

Mati Afriat (`mat6784`) — Projet Long SHDL

13 avril 2026

Contexte

Responsable du **simulateur de circuits logiques** (axe *simulation*, branche **simulation**). Objectif sprint 1 : poser un modèle objet capable de représenter portes combinatoires et bascules séquentielles, calculer leur état ternaire (UP/DW/ND) et propager les valeurs jusqu'à stabilisation. Exploration de deux modélisations en parallèle avant de choisir celle qui sera maintenue.

Activités réalisées

- **Modélisation par Lien** (`tests projet long/simulateur/`) : fil **Lien** portant un **Etat** {UP, DW, ND}, composants abstraits avec entrées/sorties (**And**, **Or**, **Non**, **Duplicateur**, **Porte**), regroupement par **TableauLien** et **Module**.
- **Bascules séquentielles** : **Bascule**, **BasculeD**, **BasculeClock** validées par les scripts `testBascule/testBasculeD/`.
- **Moteur de simulation** : **File/FileListe** pour l'ordonnancement des recalculs.
- **Modélisation alternative par Connecteur** (`simulateur/`) : interfaces **Connecteur/Composant/Structure**, **DicoConnecteur**, **LienVecteur**, **Multiplicateur**, **BouttonEntree**. Approche plus abstraite pensée pour les vecteurs et la composition.

Résultats

- **Modèle Lien fonctionnel** : 5 classes testées en JUnit (**AndTest**, **OrTest**, **NonTest**, **LienTest**, **TableauLienTest**, **DuplicateurTest**), bascule D validée par un script de table de vérité.
- **18 fichiers Java** dans `simulateur/` (approche Connecteur) + **27 fichiers Java** dans `tests projet long/simulateur/` (approche Lien).
- Le modèle Lien est jugé suffisant pour le sprint 2 sur la combinatoire scalaire ; l'approche Connecteur reste comme piste pour les vecteurs et l'appel de module.

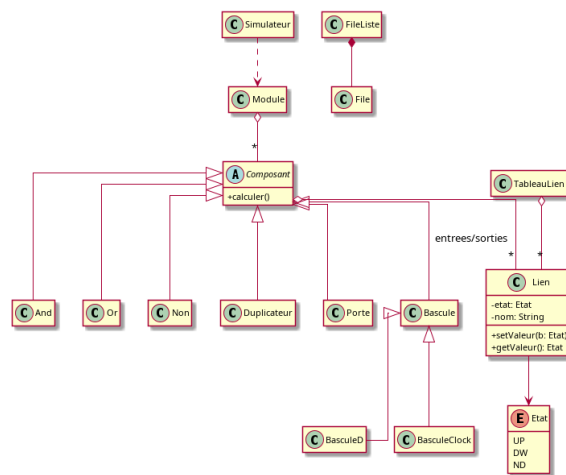


Figure — Diagramme de classes du modèle Lien (source `mati-sprint1-classes.puml`).

Difficultés et réussites

Difficulté : la **convergence des circuits séquentiels**. Les bascules introduisent des boucles de rétroaction ; il a fallu définir un ordre de recalcul stable et un état ND pour les liens non encore définis, ce qui a guidé le choix de la file d'événements. Réussite : la base combinatoire est très simple à tester et a servi de socle pour les bascules sans modification ultérieure.

Manuel utilisateur

Pré-requis : JDK 21+, JUnit 4 et Hamcrest. Tests : `javac -cp lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar 'tests projet long/simulateur/*.java'` puis `java org.junit.runner.JUnitCore simulateur.AndTest`. Scripts de bascule : `java simulateur.testBasculeD`. Code sur la branche **simulation**.